BigDecimal v1.2 Add-in

Functional Understanding Document

Version 1.0

Table of Contents

List of Tables

List of Figures

1 Introduction

Welcome to BigDecimal, a a high-performance, arbitrary-precision decimal library designed to break the 15-digit precision barrier of standard Excel calculations. Heavily inspired by the robust BigDecimal implementations found in enterprise languages like Java, this add-in brings industrial-grade numerical accuracy directly to your spreadsheets.

Whether you are performing complex financial reconciliations or deep scientific research, BigDecimal ensures that your results are never compromised by floating-point errors or rounding limitations.

1.1 Why BigDecimal?

Standard Excel uses the IEEE 754 floating-point standard, which is excellent for speed but can introduce "silent" errors in very large or very small numbers. BigDecimal for Excel replaces this with an exact-value representation, allowing you to define the exact scale and precision required for your specific use case.

2 Installation of BigDecimal v1.2.xlam

Follow the steps below to install the **BigDecimal** add-in and integrate it with Microsoft Excel. These steps ensure the library is correctly registered and available for use within the VBA Editor.

2.1.1 Downloading the BigDecimal Add-in

To install the Add-in, download the **BigDecimal v1.2.xlam** file from the GitHub repository and move it to the local Microsoft Add-ins directory:

C:\Users\[*YourUsername*]\AppData\Roaming\Microsoft\AddIns

Note: Replace [YourUsername] with your active Windows login name.

Quick Access Shortcut:
Alternatively, you can access this folder directly by pasting the following environment variable into the File Explorer address bar:

%appdata%\Microsoft\AddIns

Once the folder is open, drag and drop the BigDecimal v1.2.xlam file from your **Downloads** folder into the **Addins** folder as in Figure 2-1.

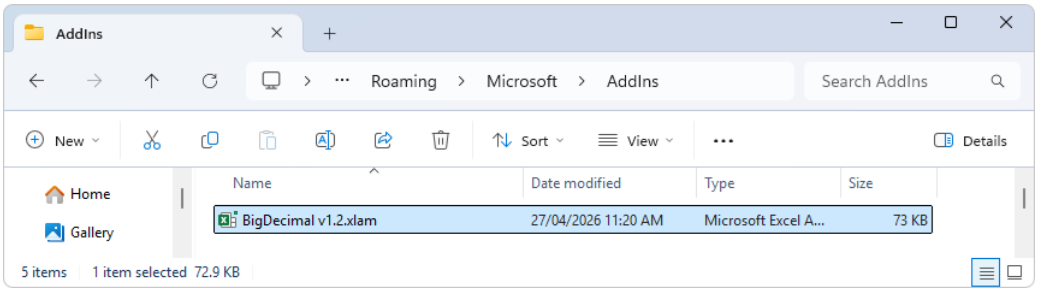

Figure 2-1: Excel Add-ins Folder

2.1.2 Important: Unblocking the Add-in

Because the file was downloaded from an external source (GitHub), Windows will apply a 'Mark of the Web' security attribute, which prevents the Add-in from executing macros. Even if the file is placed in the correct directory, it will not function until it is manually unblocked.

To unblock the Add-in:

1. Navigate to the %appdata%\Microsoft\AddIns folder
2. **Right-click** the BigDecimal v1.2.xlam file.
3. Select **Properties**.
4. At the bottom of the **General** tab, look for **Security**.
5. Check the **Unblock** box and click **Apply**.

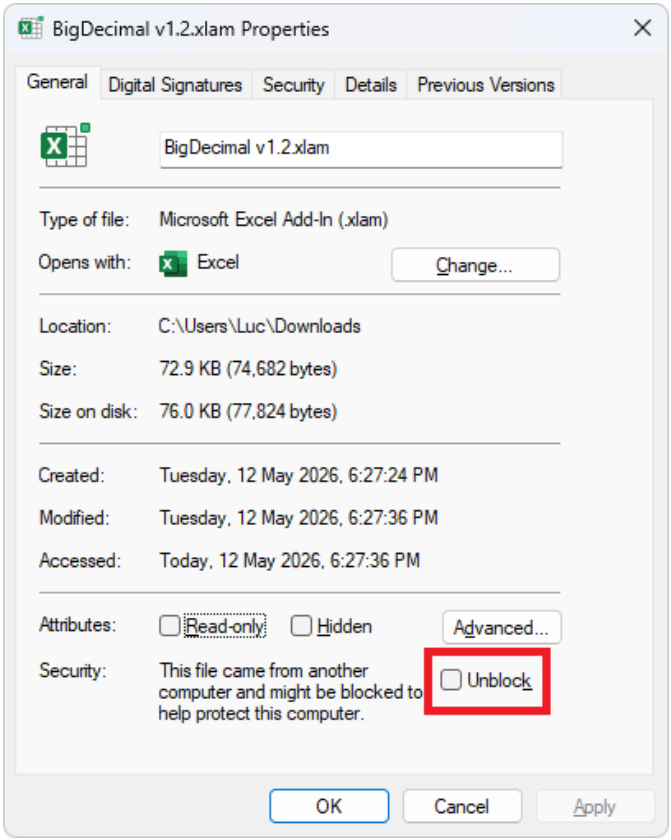

Figure 2-2: Unblock Downloaded BigDecimal Add-in

3 Working with BigDecimal

Before the **BigDecimal** class can be utilized in your VBA project, it must be properly initialized within the VBA environment. This involves linking the Add-in to your specific workbook and ensuring the VBA compiler can access the library's properties and methods.

3.1 Referencing BigDecimal in VBA

Once the Add-in is installed and unblocked, you must establish a reference to the library within your specific VBA Project. This allows the VBA compiler to recognize the **BigDecimal** class and provide IntelliSense support while coding.

Follow these steps to enable the reference:

1. Open Excel.
2. Go to the **Developer** tab (If you don't see it, right-click any ribbon tab > Customize the Ribbon > Check "Developer").
3. Click **View Code**.
4. Go to the **Tools** menu > **References..** and locate BigDecimal_v1_2 and select it as in Figure 3-1. A tick mark will appear next to BigDecimal_v1_2.
5. Click **OK**.

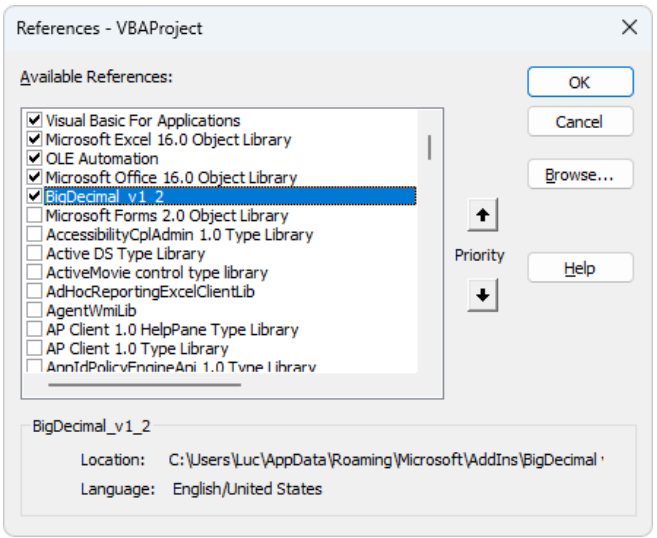

Figure 3-1: Adding BigDecimal_v1_2 as a Reference

At this stage the add-in is now available to use in your VBA Project.

4 BigDecimal "Quick Start" Guide

BigDecimal is designed to work seamlessly within the Excel environment, its implementation respects the standard VBA architectural patterns used by Microsoft Office. To ensure maximum compatibility and performance across different workbooks, the library utilizes a

controlled instantiation model. This approach prevents common reference errors and ensures that every BigDecimal object is correctly initialized with the high-precision engine.

4.1 Instantiating the BigDecimal Class

In standard object-oriented programming, you would typically instantiate a class using the `New` keyword. However, due to the **`PublicNotCreatable`** nature of classes within an Excel Add-in environment, the BigDecimal class cannot be created directly by external projects.

To bypass this limitation, the Add-in provides a global factory function: `New_BigDecimal()`. This function serves as the entry point for object creation, allowing you to initialize a new BigDecimal instance. It optionally accepts a string parameter to set the initial value of the number, ensuring precision is maintained from the moment of instantiation.

```
Public Function New_BigDecimal(Optional StrValue As String = "") As
BigDecimal
```

4.2 Implementation Example: Basic Addition

The following example, adapted from the **BigCalculator** source code, demonstrates how to instantiate two BigDecimal objects from user input, perform basic validation, and output the result of an addition operation.

```
Private Sub Add_Click()
    Dim Num1 As BigDecimal
    Dim Num2 As BigDecimal

    With BigDecimalForm
        ' Instantiate objects using the New_BigDecimal factory function
        Set Num1 = New_BigDecimal(.Num1.Value)
        Set Num2 = New_BigDecimal(.Num2.Value)

        ' Validate inputs and check for successful instantiation
        If .Num1.Value = "" Then
            ErrBox "Number 1 is empty!", "Input Error"
            Exit Sub
        ElseIf .Num2.Value = "" Then
            ErrBox "Number 2 is empty!", "Input Error"
            Exit Sub
        ElseIf Num1 Is Nothing Then
            ErrBox "Number 1 is invalid!", "Validation Error"
            Exit Sub
        ElseIf Num2 Is Nothing Then
            ErrBox "Number 2 is invalid!", "Validation Error"
            Exit Sub
        End If

        ' Perform addition and return the result as a string
        .Answer.Value = Num1.Add(Num2).StrValue
    End With
End Sub
```

4.3 Language Limitations: Operator Overloading

A notable constraint of the VBA environment is the lack of **Operator Overloading**. In many modern languages, you can redefine standard operators (such as +, -, *, or /) to work directly with custom classes. Ideally, one would use a syntax like Set Result = Num1 + Num2; however, this is not supported in VBA.

Consequently, there is no direct workaround for this language limitation. All arithmetic operations must be performed by calling the explicit methods of the BigDecimal class. While this requires a slightly more verbose coding style, the method names are intuitive and follow standard mathematical naming conventions.

5 Functional Specifications

This section provides a detailed technical breakdown of the public classes, methods, and properties that comprise the **BigDecimal** framework. These components form the core engine of the library, enabling the programmatic initialization of high-precision variables, the execution of complex mathematical operations, and the fine-tuning of calculation parameters such as scale and rounding. By leveraging these members, developers can extend the numerical capabilities of their Excel applications far beyond the standard limits of floating-point arithmetic.

5.1 BigDecimal Methods

The following table outlines the comprehensive suite of arithmetic and utility methods supported by the **BigDecimal** class, designed to provide consistent results for high-precision computational tasks.

Method	VBA Syntax (BigDecimal)	Notes
Add	`Num1.Add(Num2)`	Returns the sum of Num1 and Num2.
Subtract	`Num1.Subtract(Num2)`	Returns the difference (Num1 - Num2).
Multiply	`Num1.Multiply(Num2)`	Returns the product of Num1 and Num2.
Divide	`Num1.Divide(Num2)`	Returns the quotient; precision depends on settings.
Modulus	`Num1.Modulus(Num2)`	Returns the remainder of the division.
SquareRoot	`Num1.SquareRoot()`	Calculates the square root to current precision.
Logarithm	`Num1.Logarithm()`	Calculates the Natural Logarithm (*ln*).
Exponential	`Num1.Exponential()`	Calculates the Natural Exponent (*e^x*).
Factorial	`Num1.Factorial()`	Calculates the product of all positive integers.
Absolute	`Num1.Abs()`	Returns the absolute (non-negative) value.
Negative	`Num1.Negative()`	Returns the value multiplied by -1.
Whole	`Num1.Whole()`	Returns the integer part without rounding.
Fraction	`Num1.Fractional()`	Returns only the decimal/fractional part.
Round	`Num1.Round(Scale)`	Rounds to a specified number of decimal places.
Trim	`Num1.Trim()`	Trim leading and trailing 0
Compare	`Num1.Compare(Num2)`	Returns -1 (less), 0 (equal), or 1 (greater).

Table 5-1:BigDecimal Methods

5.2 BigDecimal Properties

The **BigDecimal** class includes a set of public properties that provide critical insights into the state and value of your numerical objects. Unlike standard numeric types in VBA, which require complex parsing to analyze, these properties allow you to directly access the internal characteristics of a number—such as its string representation, its current scale (the number of digits after the decimal point), and its sign. By utilizing these properties, developers can easily integrate high-precision results into the Excel UI or use them as conditional logic in complex financial and scientific algorithms.

Property	Access	Notes
.Value	Read/Write	Gets or sets the **BigDecimal** object itself.
.StrValue	Read/Write	Gets or sets the value via a **String**. Ideal for updating the object from a TextBox.
.LngValue	Read/Write	Gets or sets the value via a **Long**. Overwrites the object with a whole number.
.DblValue	Read/Write	Gets or sets the value via a **Double**. Useful for quick conversion from standard variables.
.MaxScale	Read/Write	Gets or sets the number of decimal places. Setting this can truncate or pad the number.
.Pi	Read Only	Returns the constant Pi as a high-precision **BigDecimal** object.

Table 5-2: BigDecimal Properties

5.2.1 Effortless Precision with Auto-Trimming

The **BigDecimal** class is designed to deliver clean, professional output without the need for manual string manipulation. While the internal object may retain leading and trailing zeros to preserve the integrity of its data structure, the **`.StrValue`** property handles the cleanup for you. Every time you access the value via this property, the library automatically trims any non-significant zeros, ensuring your results are presented in their most concise and readable form while maintaining the high-precision accuracy you require.

5.2.2 Mutable vs. Immutable Properties

Most properties in the **BigDecimal** class are **Mutable**, meaning you can change the value of an existing object without needing to create a new one using the factory function.

- **Direct Modification:** By setting .StrValue or .DblValue, you can reuse an existing BigDecimal variable and update its internal data instantly.
- **Precision Control:** Manually setting the .Scale allows you to force a specific decimal length on an existing number, which is useful for formatting financial data (e.g., forcing 2 decimal places).
- **The Pi Exception:** Unlike the other properties, **Pi** is a constant. It cannot be overwritten, as it represents a fundamental mathematical value. Attempting to set Num1.Pi = 3 will result in a compilation error, ensuring the integrity of your geometric calculations.

6 Conclusion

Thank you for choosing **BigDecimal**.

The goal of this library is to provide a robust, high-precision numerical engine that bridges the gap between standard VBA data types and the rigorous accuracy required for modern financial and scientific applications.

I hope the current feature set—from the intelligent **auto-trimming** logic to the definitive precision control of **MaxScale**—empowers you to build more reliable and professional tools. As the development of this class continues, I am committed to maintaining this balance of mathematical integrity and developer-friendly flexibility.